

# 테스트 세션 사전 공유

이번 세션은 테스트 문법이나 작성법을 처음부터 설명하는 자리는 아닙니다.

우리도 필요할 때 테스트를 작성하고 있습니다. 다만 일정이나 실행 환경, 테스트 데이터 준비 같은 현실적인 이유로 테스트가 뒤로 밀리는 경우가 있고, 무엇을 어디까지 테스트할지 판단하기 어려울 때도 있습니다.

이번에는 이런 상황에서 **어떤 테스트를 남기는 것이 실제로 도움이 되는지**를 우리 코드와 사례를 보면서 이야기해보려고 합니다.

## 왜 이 이야기를 하나

기능을 수정한 뒤 변경한 부분은 직접 확인했지만, 예상하지 못한 곳에서 문제가 생기는 경우가 있습니다.

- 수정한 기능은 정상인데 연관된 기능이 달라진다.
- 배포 전에 같은 시나리오를 여러 번 손으로 확인한다.
- 영향 범위를 확신하기 어려워 기존 코드를 건드리기 부담스러워진다.
- 문제가 나중에 발견되면 어느 변경에서 시작됐는지 다시 추적해야 한다.

테스트가 모든 버그를 막아주지는 않습니다. 대신 변경 전후에 지켜져야 할 동작을 반복해서 확인하고, 문제가 생겼을 때 원인 범위를 좁히는 데 도움을 줍니다.

이번 세션에서 확인하려는 것도 이 지점입니다.

테스트 코드 작성이 개발 속도를 늦추는가?

테스트를 작성하는 시간뿐 아니라 변경 결과를 확인하고 문제를 추적하는 시간까지 포함하면 어떨까?

## 우리 레포에 이미 있는 예시

`test/customer-order/create-customer-order.e2e-spec.ts` 에 있는 테스트입니다.

```
it('cardCompany가 있고 installment가 없으면 00을 저장한다', async () => {
  const createDto = createOrderDto({
    orderAmount: 10000,
    payment: { amount: 10000, cardCompany: '신한카드', approvedAt: new Date() },
  })

  const response = await createOrder(createDto).expect(201)

  const payment = await paymentRepo.findOne({ where: { orderId: response.body.id } })
  expect(payment?.installment).toBe('00')
})
```

이 테스트가 남기는 것은 단순히 코드 한 줄의 결과가 아닙니다.

카드사 정보는 있지만 할부 정보가 없다면 일시불( 00 )로 저장한다.

주문 생성 로직이나 결제 저장 방식이 바뀌더라도 이 규칙은 유지되어야 한다는 것을 확인합니다. 테스트 이름만으로도 어떤 조건과 결과를 지키려는지 알 수 있습니다.

이런 테스트가 필요한 이유는 코드를 많이 검증하기 위해서라기보다, **변경되어서는 안 되는 규칙을 분명하게 남기기 위해서** 라고 생각합니다.

다만 테스트에 적힌 기대 결과가 잘못되면 잘못된 동작도 정답으로 남을 수 있습니다. 테스트 작성 자체보다 “이 조건에서 어떤 결과가 맞는가”를 확인하는 일이 먼저인 이유입니다. AI가 테스트 초안을 작성할 때도 이 판단은 사람이 해야 합니다.

## 모든 코드를 테스트할 필요는 없다

---

테스트가 많다는 사실만으로 변경이 안전해지는 것은 아닙니다. 우리 서비스에서는 실패했을 때 영향이 큰 규칙부터 확인하는 편이 현실적입니다.

예를 들면 다음과 같습니다.

1. 결제·취소·환불과 주문 상태 전이
2. 금액·수량·매출·시간대 계산
3. 매장과 사용자 사이의 데이터 접근 권한
4. 실제로 발생했던 버그의 재현 케이스
5. DB나 외부 시스템과 연결되는 경계

반대로 프레임워크가 이미 보장하는 동작이나 구현 내부의 사소한 호출 방식은 우선순위가 낮을 수 있습니다.

핵심은 커버리지 수치를 높이는 것이 아니라 다음 질문에 답하는 것입니다.

이 변경에서 가장 걱정되는 동작은 무엇이고, 그것을 지금 어떻게 확인하고 있는가?

## 세션 목표

---

이번 세션의 목표는 모든 코드에 테스트를 붙이는 규칙을 만드는 것이 아닙니다. 우리 업무에서 테스트를 작성하는 편이 더  
이득인 지점을 함께 구체화하는 것입니다.